AMRITA VISHWA VIDYAPEETHAM

Version-Aware RAG for Reliable Network CLI Configuration Synthesis

Department of Computer Science & Engineering

Student Members [Batch ID: 66]

1:Aryan Madhusudhanan – CH.SC.U4CSE23207
2:Srivishnu T – CH.SC.U4CSE23249

Amrita School of Computing

Amrita Vishwa Vidyapeetham, Chennai Campus

Guide Details

V Nivethitha
Assistant Professor - CSE

Abstract

Abstract

Background Of Study

Problem Statement

Scope of Work

Significance

Objective

Literature Review

Research Gaps

Architecture Diagram

Proposed Methodology

Results

Gantt Chart

Reference

The integration of Large Language Models (LLMs) into Intent-Based Networking promises to bridge the gap between high-level operational goals and low-level CLI configurations. However, LLMs frequently suffer from hallucinations and version pollution, where incompatible commands are erroneously mixed. Here we present a Version-Aware Retrieval-Augmented Generation (RAG) architecture for CLI synthesis in heterogeneous environments (Cisco IOS-XE and Juniper JunOS). Our approach leverages Hierarchical Knowledge Grounding, where LLM generation is constrained by metadata-filtered vendor documentation with OS version locking. We introduce a Syntactic Verification Gate using Batfish static analysis. Our evaluation on 23 test intents demonstrates zero hallucinations (0% vs 4.3% baseline) with 87% Syntactic Validity. We identify a 17.4% Critical Failure Rate — silent, semantic errors that pass syntactic checks but fail operational intent. We conclude that current AI-driven synthesis is unsuitable for autonomous deployment and propose a Grounding Audit framework that surfaces authoritative citations for human expert review.

Introduction - Background of Study

Abstract

Background Of Study

Problem Statement

Scope of Work

Significance

Objective

Literature Review

Research Gaps

Architecture Diagram

Proposed Methodology

Gantt Chart

Reference

- **The IBN Paradigm**: Intent Based Networking (IBN) aims to automate the network lifecycle by translating high-level business policies into device-specific CLI configurations.
- **The Northbound Challenge**: Manual translation in heterogeneous environments (multi-vendor, multi-OS) remains a primary source of operational complexity.
- **The LLM Opportunity**: Large Language Models (LLMs) offer a natural language interface for adaptive CLI synthesis, promising to replace rigid, template-based engines.
- **Current State of Industry**: Most current AI implementations rely on a "Human in the Loop" (HITL) model but often lack the technical grounding to make that human oversight effective.

Introduction - Problem Identification & Statement

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Problem Identification:

The Reliability Crisis: Human error remains the primary cause of network outages, as generalist AI models exhibit a **36% syntax failure rate** when translating high-level intent to low-level CLI.

Technical Failure Modes:

- **Syntactic Fabrication**: Generation of "hallucinated" commands that do not exist in any valid vendor documentation.
- **Version Pollution**: Mixing of incompatible commands across OS versions due to "Syntax Drift" from firmware updates.
- **Semantic Drift**: Syntactically "perfect" configurations that fail to implement the user's operational intent, leading to silent failures.
- **The Verification Gap ("Safety Washing")**: "Human in the Loop" often serves as a liability disclaimer rather than a safety mechanism because reviewers lack the cognitive tools to audit complex configurations in real-time.
- **The "Last Mile" Grounding Gap**: Current IBN frameworks struggle with a lack of a deterministic link to authoritative vendor documentation for CLI execution.

Introduction - Problem Identification & Statement

Abstract

Background Of Study

Problem Statement

Scope of Work

Significance

Objective

Literature Review

Research Gaps

Architecture Diagram

Proposed Methodology

Results

Gantt Chart

Reference

Problem Statement:

Generating reliable network configurations from high-level intent remains a significant challenge, as both human operators and LLM-based systems are prone to errors. Generalist LLMs frequently produce hallucinated or syntactically invalid commands, introduce version inconsistencies across network operating systems, and exhibit semantic misalignment with the intended operational goals. Furthermore, existing approaches lack deterministic grounding in authoritative vendor documentation and fail to provide robust, real-time verification mechanisms, resulting in configurations that are difficult to trust for autonomous deployment.

Introduction - Scope of Work

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

- **Heterogeneous Environments**:
  Focuses on **Cisco IOS-XE (17.6)** and **Juniper JunOS (22.4)**.

- **Protocol Focus**:
  Primarily evaluates **OSPF** and **BGP** routing, alongside ACL and VLAN management

- **Toolchain**:
  Utilizes Llama 3.1 (8B) for generation, ChromaDB for vector storage, and **Batfish** for static analysis.

- **Validation Constraints**:
  Synthesis is validated against a Batfish parsable subset; complex BGP attributes rely on secondary regex or manual review.

Significance

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Survey
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

I. Societal and Technological Importance

- **Zero Hallucination Rate:** Achieved 0% syntax fabrication via authoritative grounding.
- **Audit-Ready Synthesis:** Shifts from probabilistic guesses to verifiable, grounded CLI.
- **Reliability-First AI:** Solves the 36% syntax failure rate of generalist models.
- **Version-Aware Logic:** Eliminates "Syntax Drift" through strict metadata filtering.

II. Stakeholders

- **NOC Engineers:** Rapid access to accurate syntax for faster site configuration.
- **Enterprise Architects:** Ensured compliance through version-locked OS filters.
- **Security Reviewers:** Reduced "Last Mile" search effort via the Audit Trail.
- **Multi-Vendor Teams:** Standardized intent across Cisco and Juniper ecosystems.

III. Applications

- **Legacy Hardware Support:** Bridges knowledge gaps for older, non-training set data.
- **Context-Aware CLI:** Generates site-specific scripts using local documentation.
- **Syntactic Verification:** 87% syntactic validity via the Batfish Verification Gate.
- **Dynamic Knowledge Injection:** Real-time updates for patch notes and vendor advisories.

Objective

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Phase 1: Knowledge & Static Verification	Phase 2: Dynamic Digital Twin (GNS3)	Phase 3: Feedback-Driven Self-Correction
Objective 1: Develop a Version-Aware RAG architecture using metadata filtering to lock the LLM into specific vendor documentation.	**Objective 1:** Integrate GNS3 to create a high-fidelity Digital Twin for dynamic, real-time configuration testing.	**Objective 1:** Develop a Structured Log Parser to translate raw GNS3/Batfish error logs into actionable feedback for the LLM.
Objective 2: Eliminate "Syntax Fabrication" by achieving a 0% hallucination rate through authoritative grounding.	**Objective 2:** Execute synthesized CLI in virtualized topologies to validate operational states and protocol stability.	**Objective 2:** Implement a Contextual Retry Mechanism that feeds specific failure reasons back to the engine for targeted "patching".
Objective 3: Utilize Batfish as a deterministic Syntactic Verification Gate to catch syntax errors before simulation.	**Objective 3:** Detect "silent" semantic errors, such as routing loops, that static analysis alone cannot identify.	**Objective 3:** Optimize Prompt Refinement to guide the LLM in fixing specific configuration blocks without full re-synthesis.
Objective 4: Provide a Grounding Audit Trail with command-level citations to simplify expert human review.	**Objective 4:** Ensure behavioral compliance by comparing GNS3 simulation outcomes against the original intent.	**Objective 4:** Reduce the 17.4% Critical Failure Rate (CFR) toward 0% through iterative, feedback-led refinement.

Literature Review

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Paper No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
1	Bringhenti et al. (2024)	Automation for Network Security Configuration	ACM Computing Surveys	Formal verification, MaxSMT, ILP optimization.	Categorizes formal policy translation to NSF rules.	Lacks a Syntactic Validator for vendor-specific CLI tokens.
2	Boutouchent et al. (2024)	AMANOS: An Intent-Driven Management and Orchestration System	IEEE Communications Magazine	NLU, Ontology-based models, Machine Learning.	Proposes closed-loop orchestration for 5G Core.	Fails to address physical hardware Syntax Drift.
3	Njah et al. (2024)	An AI-Driven Intent-Based Network Architecture	IEEE Communications Magazine	LLM fine-tuning (BERT), NER, MADRL.	BERT-based intent refinement and activation logic.	Targets SDN policies rather than Native CLI synthesis.
4	Kerboeuf et al. (2024)	Design Methodology for 6G End-to-End System	IEEE Open Journal of the Communications Society	Architectural blueprinting, E2E system modeling.	Outlines blueprint for 6G intent-driven automation.	Ignores Firmware-level Syntax Drift at the network edge.
5	Haj Hussein et al. (2025)	Intelligent Intent-Based Network Slicing for IoT Systems	IEEE Transactions on Network and Service Management	Literature survey, taxonomy definition.	Vision for IoT slice management and orchestration.	Overlooks the Syntactic Synthesis for multi-vendor devices.

Literature Review

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Paper No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
6	Chen et al. (2025)	Application of RAG for Industrial Knowledge Management	Computer Standards & Interfaces	RAG, BM25, Cross-encoder reranker, GPT-4.	Metadata-aware RAG for industrial technical manuals.	Generates NL text, not machine-executable code.
7	Wang et al. (2025)	Hierarchical Index Retrieval RAG-Driven Intent Translation	IEEE Transactions on Mobile Computing	RAG with HIR, Mixture of Experts (MoE).	Combined RAG with hardware manuals for policy.	Lacks a Deterministic Syntactic Validator for tokens.
8	Su et al. (2024)	Hybrid RAG-Empowered Multimodal LLM for Secure Data Management	IEEE Internet of Things Journal	Hybrid Multimodal RAG, Cross-chain technology.	Secure medical IoT data management via RAG.	Addresses data privacy, not Intent-to-CLI translation.
9	Zhang et al. (2024)	Interactive AI With RAG for Next Generation Networking	IEEE Network	Interactive AI, RAG, Chain-of-Thought (CoT).	RAG-assisted AI for network optimization problems.	Assistant for math; generates no structural CLI configs.
10	Alabbasi et al. (2025)	TeleOracle: Fine-Tuned RAG With Long-Context Support	IEEE Internet of Things Journal	RAG with SLM (Phi-2), LoRA, SelfExtend.	SLM-based RAG aligned to 3GPP for technical Q&A.	Strictly for Q&A; lacks a Synthesis Engine.

Literature Review

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Paper No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
11	Jacobs et al. (2025)	Establishing Trust for Using Natural Language for IBN	IEEE Transactions on Network and Service Management	NER (Bi-LSTM), Ambiguity Detection, Nile Policy.	LUMI pipeline for NL-to-Policy via Nile.	Compiles to intermediate languages rather than Native CLI.
12	Kou et al. (2025)	GIA: LLM-Enabled Generative Intent Abstraction	IEEE Transactions on Cognitive Communications and Networking	LLM code generation, Subgraph isomorphism.	LLM-enabled structural policy graphs for networks.	Lacks Version-Aware Grounding for specific tokens.
13	Mekrache et al. (2024)	Intent-Based Management of Next-Generation Networks	IEEE Network	LLM (CodeLlama), Few-Shot Learning, RAG.	NL-to-JSON translation for 5G descriptors.	Targets JSON objects; provides no Last-Mile syntax.
14	Orozco et al. (2025)	Intent-Based Service Provisioning for Cobots	IEEE Transactions on Network and Service Management	Intent decomposition, Hierarchical state machines.	Business intent-to-3GPP JSON object translation.	Stops at JSON level; fails to account for Syntax Drift.
15	Angi et al. (2025)	LLNet: Instructing Softwarized Devices Using SLM	IEEE Transactions on Network and Service Management	Small Language Models, QLoRA, Few-shot.	SLM-based approach for instructing programmable devices.	Targets abstract APIs; ignores CLI Syntactic Volatility.

Literature Review

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Paper No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
16	Mehmood et al. (2025)	Knowledge-Driven Intent Life-Cycle Management	IEEE Transactions on Network and Service Management	Knowledge Graph Embedding (KG2E), NER.	KGs for metadata-aware intent life-cycle.	Lacks Token-level granularity for CLI synthesis.
17	Maatouk et al. (2025)	TeleQnA: A Benchmark Dataset to Assess LLMs Telecommunications Knowledge	IEEE Network	Automated QnA generation, Text embeddings, K-Means.	Established a 10,000-question benchmark for LLM telecom proficiency.	Focuses on evaluation; lacks a **Synthesis Engine** for 'Last-Mile' CLI execution.
18	Brodimas et al. (2025)	Intent-Based Infrastructure Orchestration Using Agentic-AI	IEEE Open Journal of the Communications Society	Multi-Agent System, Model-Context Protocol.	Multi-agent framework for Infrastructure-as-Code.	Generates K8s syntax vs. Legacy Physical CLI.
19	Haj Hussein et al. (2025)	Privacy-Preserving Intelligent Intent-Based Slicing	IEEE Internet of Things Journal	Deep Q-Networks (DQNs), PPNO.	Privacy-preserving slicing for cross-domain IoT.	Lacks Low-level CLI Accuracy for edge gateways.
20	Huang et al. (2024)	LLMs for Networking: Applications & Challenges	IEEE Network	Domain-adapted fine-tuning, RAG, CoT.	Surveyed LLM techniques for capacity planning.	Lacks a Syntactic Validator for safe hardware execution.

Research Gaps

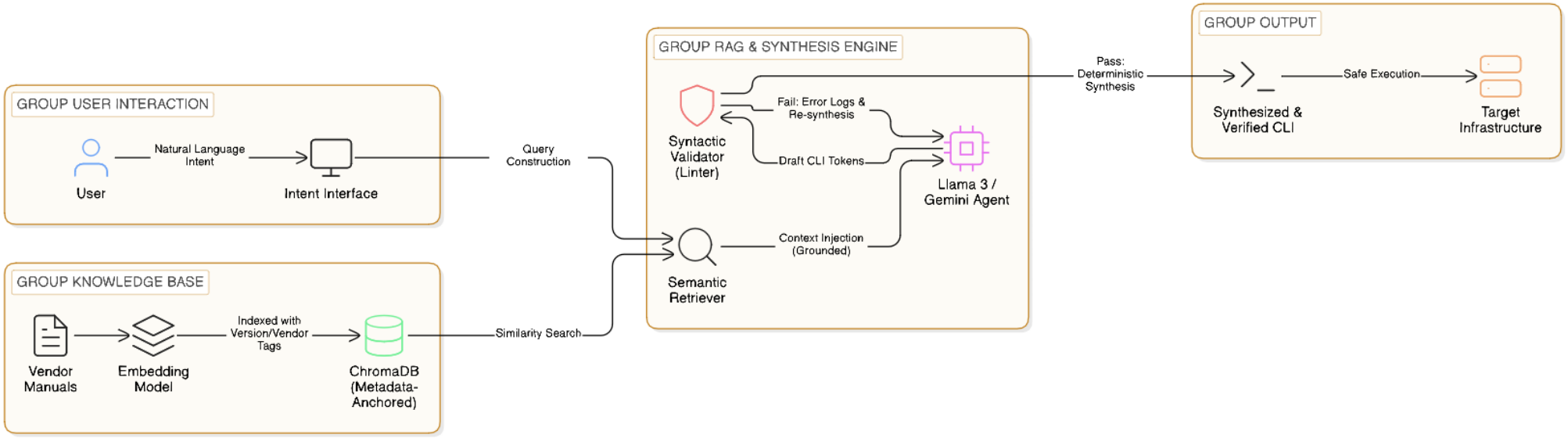
Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Current approaches to automated network configuration—including standard RAG and prompt engineering exhibit four critical limitations that prevent production-grade deployment:

- **Version Pollution**: Existing RAG systems for networking do not enforce strict OS version constraints. This leads to the "pollution" of the context window with commands that are deprecated or renamed in the target software release.
- **The "Black Box" Problem**: Generated configurations lack provenance; there is no embedded **Grounding Audit Trail** to show which manual or document page justifies a specific command. This forces human reviewers to manually search through millions of words of documentation to verify a single AI-generated line.
- **Probabilistic Fallibility**: State of the art (SOTA) models like **LLNet** rely purely on probabilistic LLM inference without a secondary **deterministic syntactic gate**. Without static analysis tools like Batfish, syntactic errors are only caught during deployment.
- **"Safety Washing" in HITL**: Industry standards often use "Human in the Loop" (HITL) as a liability disclaimer rather than a genuine safety mechanism. Reviewers currently lack the **Cognitive Compression** tools needed to effectively audit complex, multi-vendor configurations in real-time.

Architecture Diagram

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference



AMRITA VISHWA VIDYAPEETHAM

Proposed Methodology

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

1. Intent Parser
Converts natural language into structured JSON, extracting vendor (Cisco/Juniper), OS version, protocol, and parameters.

2. Version-Aware RAG
Retrieves commands from ChromaDB (10,333 chunks), filters by vendor + OS version to avoid conflicts, and uses hierarchical chunking (commands, patterns, troubleshooting).

3. Config Generator
LLM generates CLI configurations from retrieved context, with per-command citations to ensure grounding and prevent hallucinations.

4. Verification Gate
Batfish validates configs for syntax errors, undefined references, and routing issues; failed outputs are refined (up to 3 attempts).

5. Audit Trail
Provides command-level citations, reducing documentation lookup effort and enabling quick expert verification.

Implementation & Prototype Details

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

1. **Prototype Built:**
   a. Developed a fully functional local Python application integrating an LLM synthesis engine with a vector-database knowledge base.
2. **Tools & Tech Stack:**
   a. LLM Engine: Llama 3.1 (8B) via Groq .
   b. Vector Database: ChromaDB (indexed with 10,333 chunks of vendor manuals).
   c. Parsing & Logic: LangChain and Pydantic for structured intent extraction.
3. **Simulation vs. Real-Time:**
   a. Utilized Batfish Static Analysis as a deterministic simulation environment to validate routing logic and syntax without requiring live production hardware.
4. **Benchmark Dataset:**
   a. Constructed a custom dataset of 23 complex, multi-vendor intents covering OSPF, BGP, ACLs, and VLANs across Cisco IOS-XE (17.6) and Juniper JunOS (22.4).

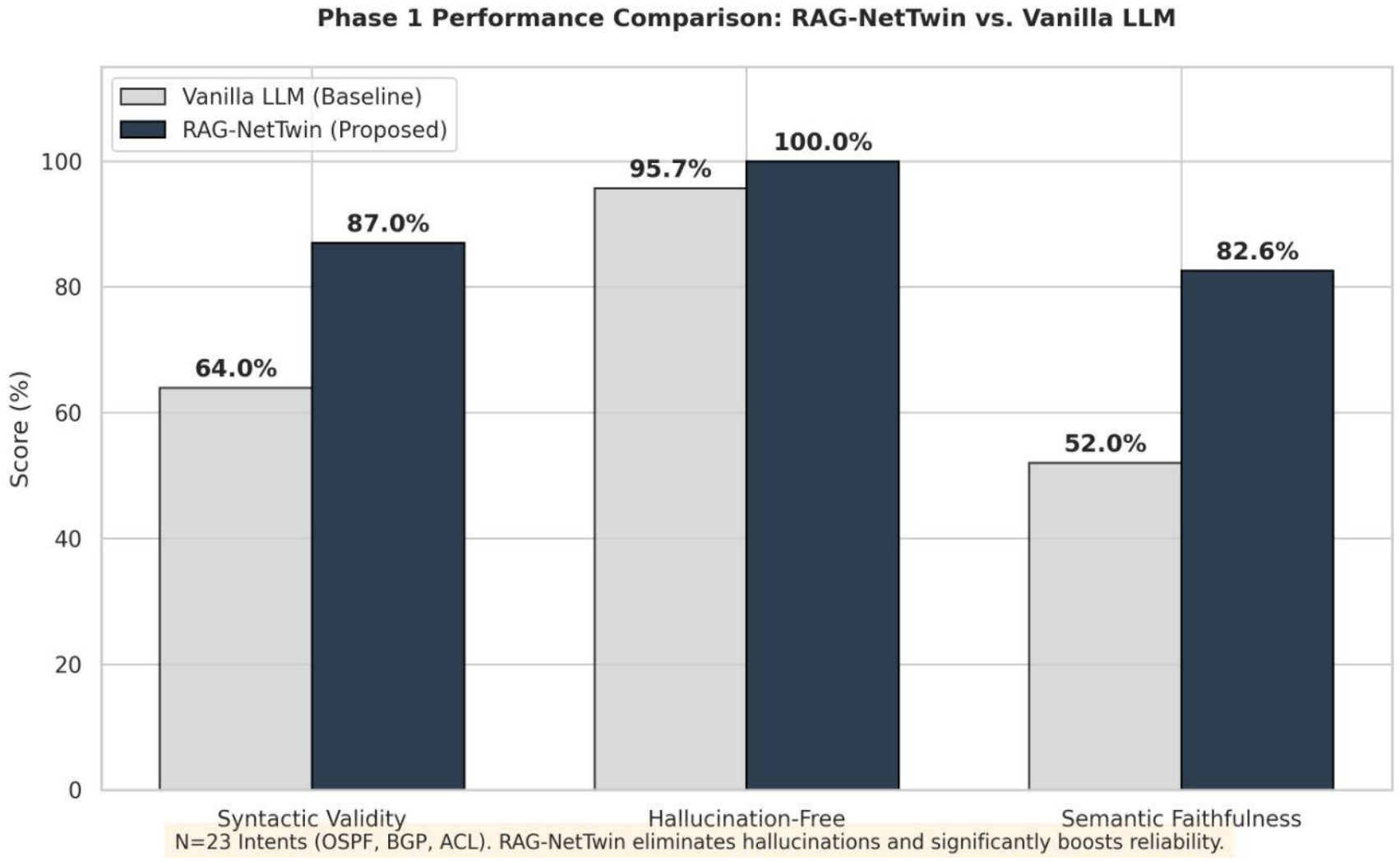
Results

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Performance Metric	RAG-NetTwin (Proposed)	Vanilla LLM (Baseline)	Improvement	Significance
Syntactic Validity	87.0%	64.0%	+23.0%	Validated via Batfish Static Analysis.
Hallucination Rate	0.0%	4.3%	-4.3%	Grounding eliminates "Syntax Fabrication."
Semantic Faithfulness	82.6%	52.2%	+30.4%	Accuracy of intent-to-CLI parameter mapping.
Critical Failure Rate	17.4%	36.0%+	-18.6%	Reduction in "Silent" semantic errors.
Grounding Audit Trail	Full Audit Trail	None	N/A	Instant source verification for human review.

Results

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference



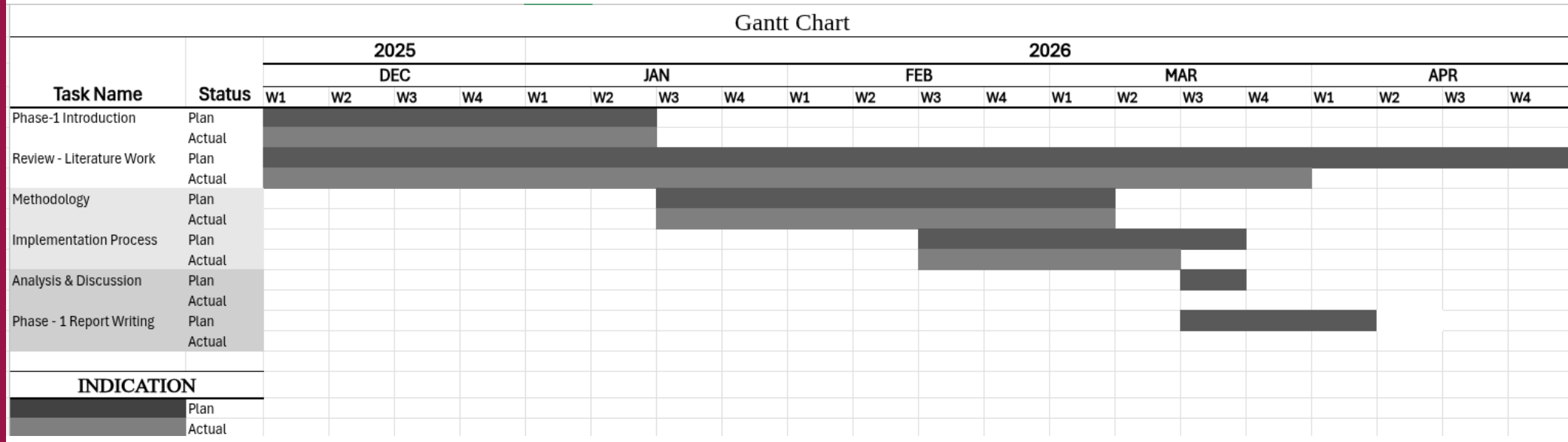
Result Analysis (Existing Systems vs. Proposed Design)

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

Feature	1. Existing System (Vanilla LLM)	2. Proposed Design (RAG-Net)
Pros	• Fast Execution: Near-instant response times. • Low Overhead: Zero database or retrieval costs. • Fluency: Naturally handles generic English intent.	• 0% Hallucinations: Grounded in verified documentation . • 87% Syntactic Validity: High success in parsable CLI . • Version Isolation: Prevents syntax drift between OS releases . • Audit Trail: Instant human verification via citations.
Cons	• Hallucinations (4.3%): Fabricates invalid commands . • Version Pollution: Mixes Cisco and Juniper syntax . • High Risk: 36%+ Critical Failure Rate (CFR) . • Black Box: No traceability to authoritative sources.	• Semantic Failure (17.4%): Missing parameters due to context limits. • Higher Latency: Retrieval and Batfish simulation overhead. • Contextual Limits: Limited by LLM context window size.

Gantt Chart

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference



AMRITA VISHWA VIDYAPEETHAM

Reference

Abstract

Background Of Study

Problem Statement

Scope of Work

Significance

Objective

Literature Review

Research Gaps

Architecture Diagram

Proposed Methodology

Results

Gantt Chart

Reference

1. D. Bringhenti et al., "Automation for Network Security Configuration: State of the Art and Research Trends," *ACM Computing Surveys*, vol. 56, no. 3, 2024.
2. A. Boutouchent et al., "AMANOS: An Intent-Driven Management and Orchestration System," *IEEE Communications Magazine*, vol. 62, no. 4, 2024.
3. Y. Njah et al., "An AI-Driven Intent-Based Network Architecture," *IEEE Communications Magazine*, vol. 62, no. 1, 2024.
4. L. Kerboeuf et al., "Design Methodology for 6G End-to-End System," *IEEE Open Journal of the Communications Society*, vol. 5, 2024.
5. D. Haj Hussein et al., "Toward Intelligent Intent-Based Network Slicing for IoT Systems," *IEEE Transactions on Network and Service Management*, vol. 22, no. 1, 2025.
6. L. Chen et al., "Application of retrieval-augmented generation for interactive industrial knowledge management," *Computer Standards & Interfaces*, vol. 91, 2025.
7. J. Wang et al., "Hierarchical Index Retrieval-Driven Wireless Network Intent Translation With LLM," *IEEE Transactions on Mobile Computing*, vol. 24, no. 2, 2025.
8. C. Su et al., "Hybrid RAG-Empowered Multimodal LLM for Secure Data Management," *IEEE Internet of Things Journal*, vol. 11, no. 18, 2024.
9. R. Zhang et al., "Interactive AI With Retrieval-Augmented Generation for Next Generation Networking," *IEEE Network*, vol. 38, no. 3, 2024.
10. N. Alabbasi et al., "TeleOracle: Fine-Tuned Retrieval-Augmented Generation With Long-Context Support," *IEEE Internet of Things Journal*, vol. 12, no. 2, 2025.

Reference

Abstract
Background Of Study
Problem Statement
Scope of Work
Significance
Objective
Literature Review
Research Gaps
Architecture Diagram
Proposed Methodology
Results
Gantt Chart
Reference

11. A. S. Jacobs et al., "Establishing Trust for Using Natural Language for Intent-Based Networking," *IEEE Transactions on Network and Service Management*, vol. 22, no. 1, 2025.
12. S. Kou et al., "GIA: LLM-Enabled Generative Intent Abstraction to Enhance Adaptability," *IEEE Transactions on Cognitive Communications and Networking*, vol. 11, no. 1, 2025.
13. A. Mekrache et al., "Intent-Based Management of Next-Generation Networks: an LLM-Centric Approach," *IEEE Network*, vol. 38, no. 2, 2024.
14. R. P. Orozco et al., "Intent-Based Service Provisioning and Closed-Loop Automation," *IEEE Transactions on Network and Service Management*, vol. 22, no. 1, 2025.
15. A. Angi et al., "LLNet: An Intent-Driven Approach to Instructing Softwarized Network Devices," *IEEE Transactions on Network and Service Management*, vol. 22, no. 1, 2025.
16. K. Mehmood et al., "Knowledge-Driven Intent Life-Cycle Management for Cellular Networks," *IEEE Transactions on Network and Service Management*, vol. 22, no. 1, 2025.
17. A. Maatouk et al., "TeleQnA: A Benchmark Dataset to Assess Large Language Models Telecommunications Knowledge," *IEEE Network*, pp. 1–1, 2025, doi: 10.1109/MNET.2025.3576035.
18. D. Brodimas et al., "Intent-Based Infrastructure and Service Orchestration Using Agentic-AI," *IEEE Open Journal of the Communications Society*, vol. 6, 2025.
19. D. Haj Hussein et al., "Privacy-Preserving Intelligent Intent-Based Network Slicing," *IEEE Internet of Things Journal*, vol. 12, no. 3, 2025.
20. Y. Huang et al., "Large Language Models for Networking: Applications, Enabling Techniques, and Challenges," *IEEE Network*, vol. 38, no. 1, 2024.

AMRITA VISHWA VIDYAPEETHAM

AMRITA VISHWA VIDYAPEETHAM

Thank You